

Exercice 1 :

1. Écrire un programme `liste_animaux.py`, qui initialise la liste et qui affiche l'ensemble des éléments ("lapin", "chat", "chien", "chiot", "dragon")
2. Afficher la liste de manière inversée.
3. Afficher la liste de manière triée.
4. Ajouter (append) l'élément `troll` dans la liste, puis supprimer l'ensemble des animaux domestiques. Afficher le résultat. Afin de réaliser la suppression, on créera une liste des animaux domestiques.

Exercice 2 :

On considère un dictionnaire de 5 étudiants avec leurs notes. Les clés sont les noms des étudiants et les valeurs des clés sont les moyennes générales obtenues après un examen.

Ecrire un programme Python qui partitionne ce dictionnaire en deux sous dictionnaires :

1. **etudiantReussis** dont les clés sont les étudiants admis et les valeurs des clés sont les moyennes obtenues (moyenne supérieurs ou égales à 10).
2. **etudiantEchecs** dont les clés sont les étudiants non admis et les valeurs des clés sont les moyennes obtenues (moyenne inférieur ou égale à 10).

Exercice 3 :

1. Définir une classe **MaClasse** possédant les attributs suivants :
 - **données** : deux attributs de classes : $x = 23$ et $y = x + 5$.
 - **méthode** : une méthode affiche contenant un attribut d'instance $z = 42$ et les affichages de y et de z .

Dans le programme principal, instanciez un objet de la classe **MaClasse** et invoquez la méthode affiche.

2. Définir une classe **Vecteur2D** avec un constructeur fournissant les coordonnées par défaut d'un vecteur du plan (par exemple : $x = 0$ et $y = 0$).

Dans le programme principal, instanciez un **Vecteur2D** sans paramètre, un **Vecteur2D** avec ses deux paramètres, et affichez-les.

3. Enrichissez la classe **Vecteur2D** précédente en lui ajoutant une méthode d'affichage et une méthode de surcharge d'addition de deux vecteurs du plan.

Dans le programme principal, instanciez deux **Vecteur2D**, affichez-les et affichez leur somme.

Exercice 4:

Définissez ci-dessous une classe **Etudiant** dont les attributs sont : prenom, nom, note_nsi, note_maths, note_phy et note_inf

Puis créez une méthode spéciale ****str(self)** qui permettra d'afficher 'Prenom NOM a obtenu une note_maths en Mathématiques, note_phy en Physique et note_inf en Informatique' en tapant **print(E)** où E est une instance de la classe Etudiant .

Exercice 5:

Concevoir une classe **EquationSecondDegre** en Python pour modéliser les équations quadratiques. Structurer la résolution du problème en appliquant les principes de la programmation orientée objet. Implémenter le calcul du discriminant et déterminer les solutions de l'équation en fonction de sa valeur.

Exercice 6:

1. Créer une classe **Animal** avec :
 - a. une méthode **__str__()** qui retourne une chaîne de la forme « un » + le nom de la classe (Animal ou une sous-classe)
 - b. une méthode **déplacer()** qui affiche (print) le nom de l'animal (donnée par la méthode précédente) suivi de « se déplace »
2. Créer une classe **AnimalVolant** héritant de **Animal** avec :
 - a. une méthode **voler()** qui affiche le nom de l'animal + « vole »
 - b. redéfinir la méthode **déplacer()** pour qu'elle appelle **voler()**
3. De même, créer une classe **AnimalNageant** avec une méthode **nager()** et une classe **AnimalMarchant** avec une méthode **marcher()**
4. Créer une classe **Pigeon** qui est un animal qui peut marcher et voler ; son mode de déplacement « par défaut » est le vol (**déplacer()** doit donc appeler **voler()**)
5. Créer un **Pigeon** et le déplacer. Tester le programme
6. Créer une classe **Canard** qui est un animal qui peut marcher, nager et voler ; son mode de déplacement « par défaut » est la nage
7. Créer un **Canard** et le déplacer. Tester le programme

Exercice 7: Un système de gestion des comptes bancaires

Chaque compte appartient à un client et peut être de deux types : **Compte Courant** ou **Compte Épargne**.

Un compte courant permet des retraits sans restriction tant que le solde est suffisant, tandis qu'un compte épargne applique des intérêts mais limite les retraits aux fonds disponibles.

Tu devras créer les classes suivantes :

- **CompteBancaire (classe mère abstraite)**, avec un titulaire et un solde.
- **CompteCourant**, qui autorise les retraits tant que le solde est positif.
- **CompteEpargne**, qui applique un taux d'intérêt et restreint les retraits.
- **Client**, qui gère plusieurs comptes bancaires.

Chaque classe devra respecter les principes d'**encapsulation**, d'**héritage**, de **polymorphisme** et d'**abstraction**.